Security Policies

Amir Masoumzadeh

CSI 424/524

Sep. 1, 2026

UNIVERSITY AT ALBANY
State University of New York

Security Policy and Mechanism

- Policy says what is, and is not, allowed
  - This defines "security" for the system
- Mechanisms (methods/tools/procedures) enforce policies

Security Policy

- Policy partitions system states into:
  - Authorized (secure)
    - These are states the system can enter
  - Unauthorized (nonsecure)
    - If the system enters any of these states, it's a security violation
- Secure system
  - Starts in authorized state
  - Never enters unauthorized state

Access Control

- Set of procedures that allow only authorized accesses happen in a system
  - It is the main mechanism of providing security in every system
  - Authorized accesses are typically specified by an *access control policy*
- Examples: access control policies in
  - Operating systems
  - Web browsers
  - Firewalls
  - Online social networks
  - ...

Types of Access Control

- Discretionary Access Control (DAC)
  - Individual user sets access control mechanism to allow or deny access to an object
- Mandatory Access Control (MAC)
  - System mechanism controls access to object, and individual cannot alter that access
- Hybrid Access Control
  - Role-Based Access Control (RBAC)
  - Attribute-Based Access Control (ABAC)
  - Relationship-Based Access Control (ReBAC)
  - ...

Access Control Matrix

	o_1	...	o_m
s_1			
s_2			
...			
s_n			

- Subjects $S = \{s_1, \ldots, s_n\}$
- Objects $O = \{o_1, \ldots, o_m\}$
  - Typically, $S \subseteq O$
- Rights $R = \{r_1, \ldots, r_k\}$
- Entries $A[s_i, o_j] \subseteq R$
  - $A[s_i, o_j] = \{r_x, \ldots, r_y\}$ means subject s_i has rights $r_x, \ldots, r_y$ over object o_j

Access Control Matrix: Example

- Processes p, q
- Files f, g
- Rights r, w, x, a, o

	f	g	p	q
p	rwo	r	rwxo	w
q	a	ro	r	rwxo

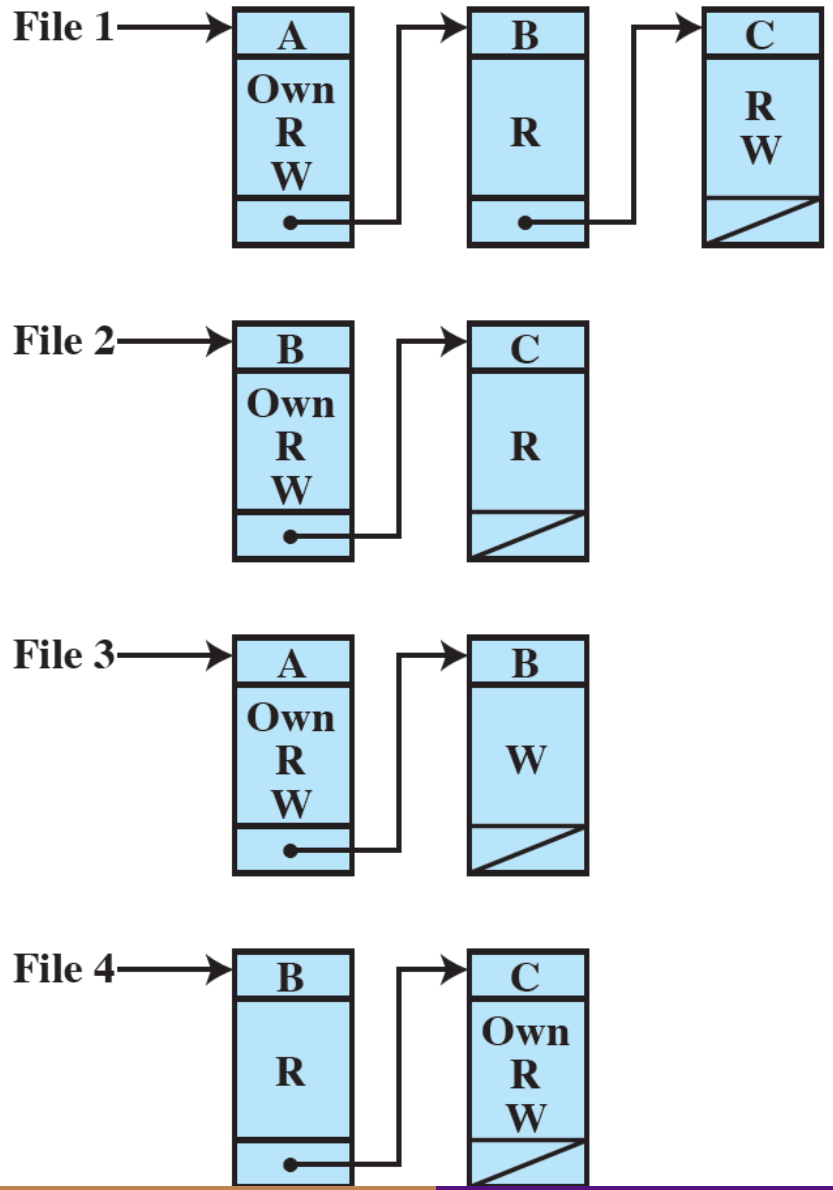
Storing Access Matrix

- How to store current state of access matrix?

		OBJECTS			
		File 1	File 2	File 3	File 4
SUBJECTS	User A	Own Read Write		Own Read Write	
	User B	Read	Own Read Write	Write	Read
	User C	Read Write	Read		Own Read Write

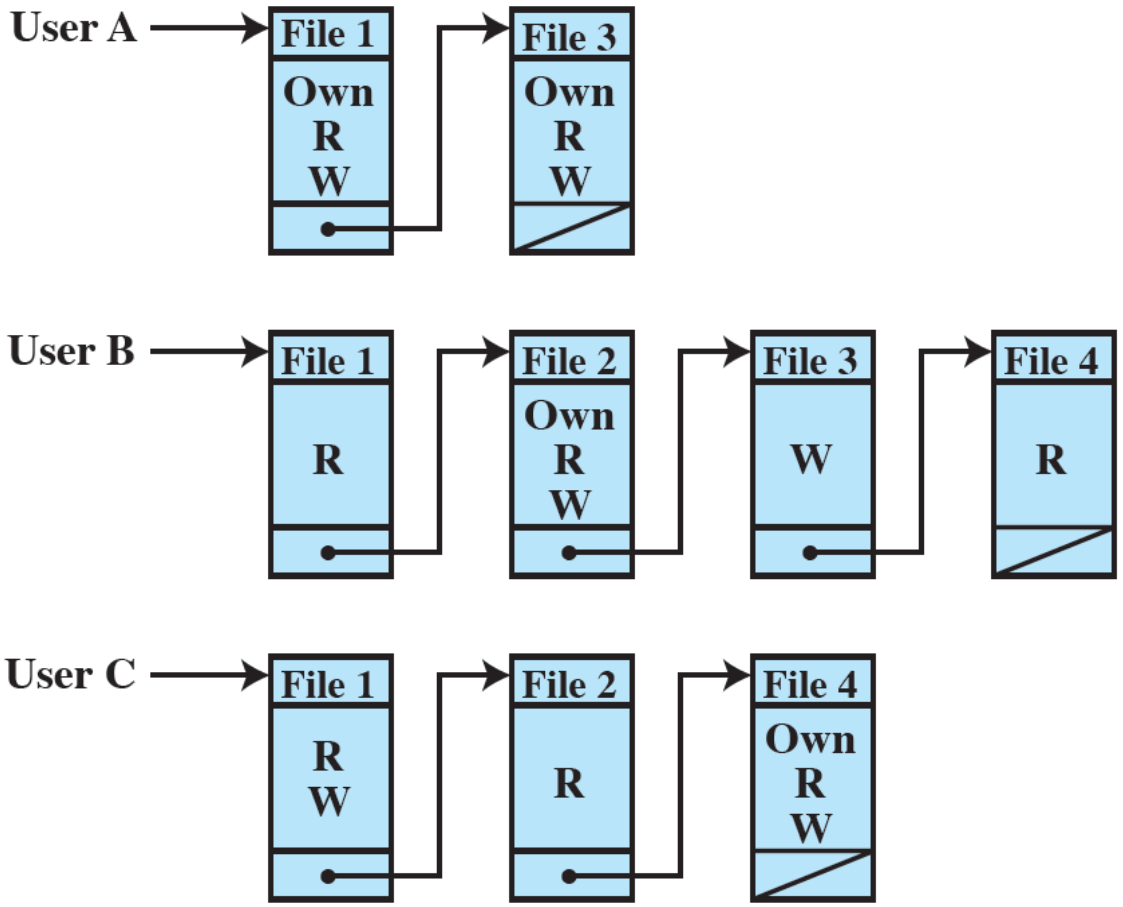
Access Control Lists (ACLs)

	OBJECTS			
	File 1	File 2	File 3	File 4
User A	Own Read Write		Own Read Write	
User B	Read	Own Read Write	Write	Read
User C	Read Write	Read		Own Read Write



Capability Lists

	OBJECTS			
	File 1	File 2	File 3	File 4
User A	Own Read Write		Own Read Write	
User B	Read	Own Read Write	Write	Read
User C	Read Write	Read		Own Read Write



Exercise 03a

Assume we have n users as subjects, m files as objects, and 4 rights. For each of the following scenarios, what is the worst case asymptotic time complexity?

1. Using capability lists, list all files to which User_A can write
2. Using access control lists, find out if User_B can write to File_1
3. Using capability lists, list all users who can delete File_2

Unix Access Control

- Users
  - Each user is assigned a user id (`uid`)
  - `root` is a superuser with `uid=0`
  - User information is stored in `/etc/passwd` file:

    ```
    john:x:1000:1000::/home/john:/bin/bash
    ```
- Groups
  - Groups help by allowing managing permissions based on groups of users too
  - Each user has a primary group (listed in `/etc/passwd`)
  - A user can belong to multiple groups (stored in `/etc/group`)

Unix: File Permissions

- Each file has a *user id* and *group id*
- File permissions
  - Mainly composed of three 3-bit parts corresponding to owner (`u`), group (`g`), and others (`o`)
  - The 3 bits correspond to read (`r`), write (`w`), and execute (`x`) permissions
- Can be also represented as octal numbers: `rwxrwxrwx = 777`
- Use `chmod` command to change file permissions (mode)
- Directory permissions interpreted differently
  - `r`: directory can be listed
  - `w`: can modify the contents of the directory (create/delete a file or a directory within it)
  - `x`: can change to that directory

Unix: Default File Permissions

- Files will be assigned their creator's `uid` and `gid` by default
- `umask`: an environment variable that filters permissions given to new files
- File permissions
  - Let `umask = 022`
  - What will be file permission if intended permission is 733?
- You can use `umask` command to query/update it

Unix: Other Useful Commands

- `whoami`: print user name
- `id`: print user and group ids
- `passwd`: change user's password
- `su`: substitute user
- `sudo`: only run one command using superuser privileges
  - Should have been added to the authorized list `/etc/sudoers`
- `chown` / `chgrp`: change owner/group of a file
- `getfacl` / `setfacl`: print or set the ACL of a file (applied in addition to regular file permissions)

Bell-LaPadula (BLP) Model

- Multi-level security model
- A mandatory access control model
- Security levels arranged in linear ordering
  - Top Secret: highest
  - Secret
  - Confidential
  - Unclassified: lowest
- Entities are assigned security levels
  - Subjects have security clearance $L(s) = l_s$
  - Objects have security classification $L(o) = l_o$

BLP: Example

security level	subject	object
Top Secret	Tom	Personal Files
Secret	Sam	Emails
Classified	Carol	Activity Logs
Unclassified	Upton	Telephone Lists

- Tom can read all files
- Carol cannot read emails or personal files
- Upton can only read telephone lists

BLP: Reading Information

- Information flows up, not down
- Simple Security Condition
  - s can read object o iff $l_o \leq l_s$ and s has discretionary read access to o
- Prevents subjects from reading objects at higher levels (no read-up rule)
- Combines mandatory control (security levels) and discretionary control (required permission)

BLP: Writing Information

- Information flows up, not down
- *-Property
  - s can write to object o iff $l_s \leq l_o$ and s has discretionary write access to o
- Prevents subjects from writing to objects at lower levels (no write-down rule)
- Combines mandatory control (security levels) and discretionary control (required permission)

Lattice (Background)

- Total order
- Partial order
  - Partially ordered set: a set of elements S and relation R, where
    - R is reflexive, antisymmetric, and transitive on the elements of S
- Hasse diagram
- Lattice is a partially ordered set S, where
  - For every $s, t \in S$, there exists a greatest lower bound
  - For every $s, t \in S$, there exists a least upper bound

BLP Model (revisited)

- Total order of classifications not flexible enough
- Expand notion of security level to include categories
  - Enforces "need to know" principle
- Security level is (classification, category-set)
  - (Top Secret, { NUC, EUR, ASI })
  - (Confidential, { EUR, ASI })
  - (Secret, { NUC, ASI })

BLP: Levels and Lattices

- (A, C) dom (A', C') iff $A' \leq A$ and $C' \subseteq C$
- Examples
  - (Top Secret, {NUC, ASI}) dom (Secret, {NUC})
  - (Secret, {NUC, EUR}) dom (Confidential,{NUC, EUR})
  - (Top Secret, {NUC}) $\neg$ dom (Confidential, {EUR})
- Security levels partially ordered
  - Any pair of security levels may (or may not) be related by dom
- Let C be set of classifications and K set of categories. Set of security levels $L = C \times K$ and dom form lattice
  - Top level (least upper bound): (max(A), C)
  - Bottom level (greatest lower bound): (min(A), $\emptyset$)

Exercise 03b

Draw a Hasse diagram for security levels formed by classifications $C < S < TS$ and categories $\{\text{US}, \text{EU}\}$.

1. How many nodes in the diagram?
2. How many links in the diagram?
3. What is the top level?
4. What is the bottom level?

BLP Access Rules (revisited)

- Simple Security Condition
  - s can read object o iff l_s dom l_o and s has discretionary read access to o
- *-Property
  - s can write to object o iff l_o dom l_s and s has discretionary write access to o

Biba Model

- Similar to security levels in BLP, we can consider *integrity levels*
- The higher the level, the more confidence that
  - a program will execute correctly
  - data is accurate and/or reliable
- Note relationship between integrity and trustworthiness
- Important: integrity levels are not security levels (as used in BLP)

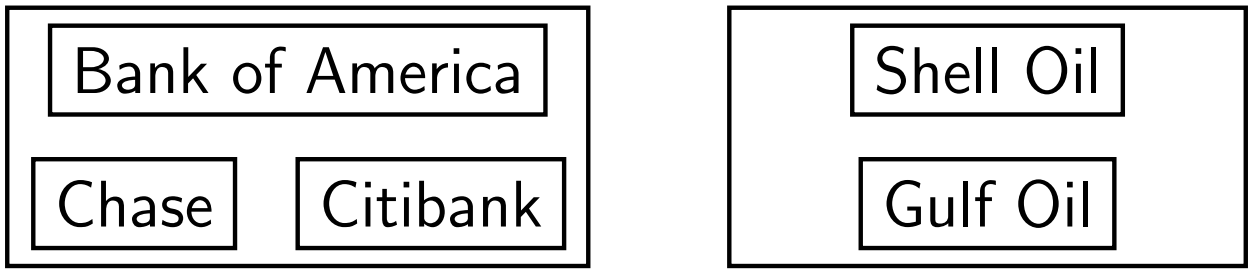
Biba Model (Strict Integrity Policy)

- Dual of Bell-LaPadulla model:
  - $s \in S$ can read $o \in O$ iff $i(s) \leq i(o)$
  - $s \in S$ can write to $o \in O$ iff $i(o) \leq i(s)$
  - $s_1 \in S$ can execute $s_2 \in S$ iff $i(s_2) \leq i(s_1)$
- You can add categories and discretionary controls to get a full dual of BLP

Chinese Wall Model

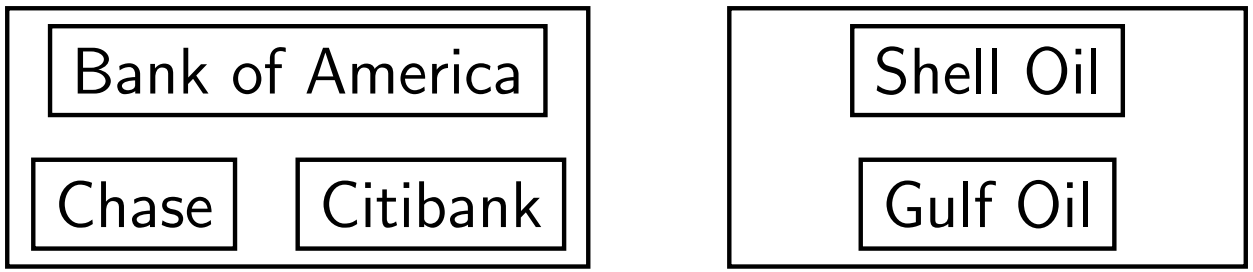
- Organize entities into "conflict of interest" classes
  - Restrict information flow between items in the same class
- Allow sanitized data to be viewed by everyone
- Model
  - Objects: items of information related to a company
  - Company dataset (CD): contains objects related to a single company
  - Conflict of interest class (COI): contains datasets of companies in competition
  - Assumption: each object belongs to exactly one COI class

CW Example: COI Classes for Banks and Oil Companies

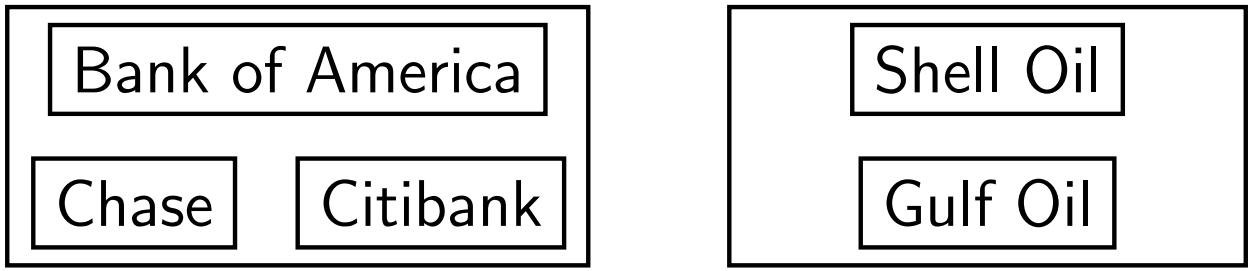


CW-Simple Security Condition

- Let $PR(s)$ be the list the previously-accessed objects by s
- s can read o iff either condition holds:
  - $\exists o' \in PR(s)$ such that $CD(o') = CD(o)$
    - Meaning s has read something in o's dataset
  - $\forall o' \in PR(s) : COI(o') \neq COI(o)$
    - Meaning s has not read any objects in o's conflict of interest class
  - o has been sanitized



CW: How About Writing?



- Alice and Bob work in the same trading house
- Alice can read Bank of America's CD
- Bob can read Citibank's CD
- Both can read Shell's CD
- Alice can write to Shell's CD
- Any problem with this setup?

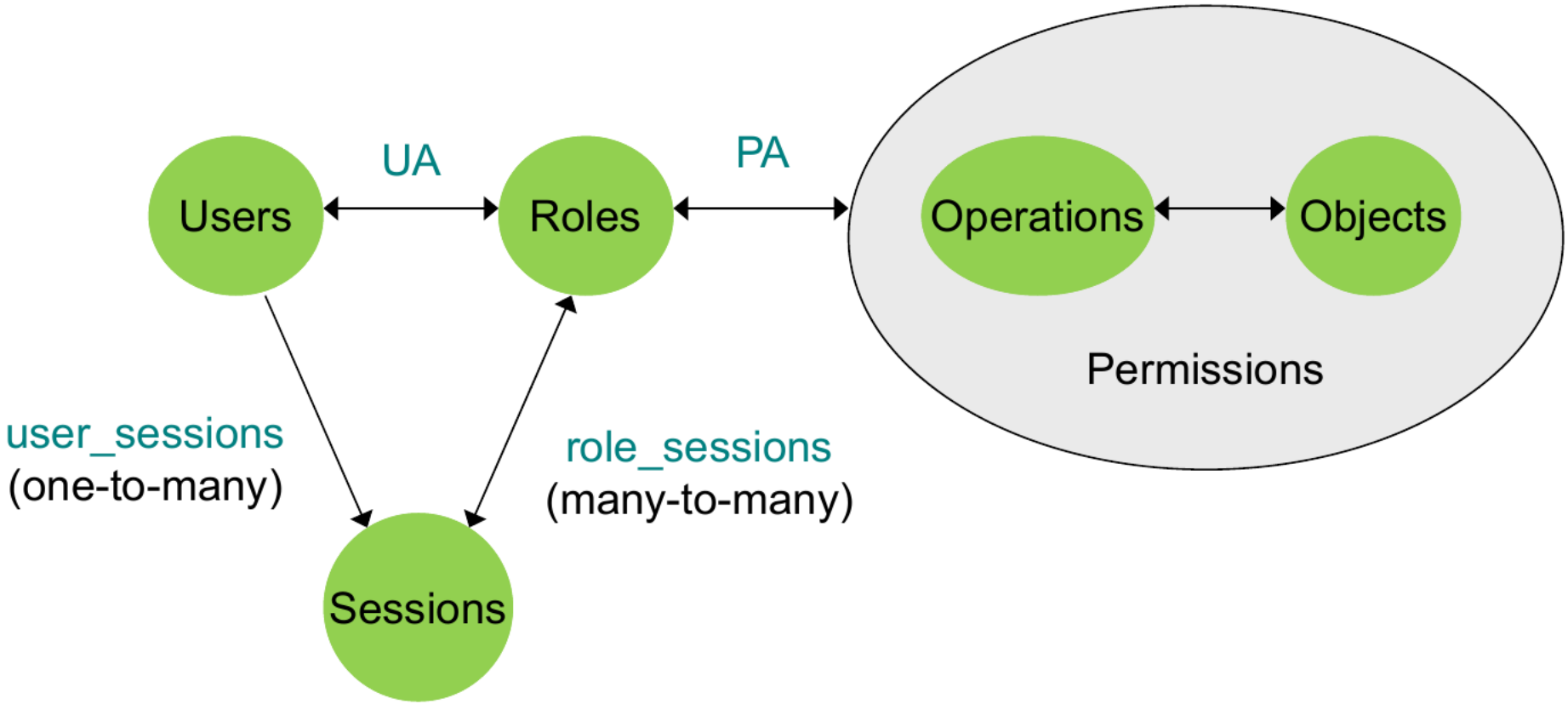
CW-*-Property

- s can write to o iff both of the following hold:
  - The CW-simple security condition permits s to read o; and
  - For all unsanitized objects o', s can read $o' \Rightarrow CD(o') = CD(o)$

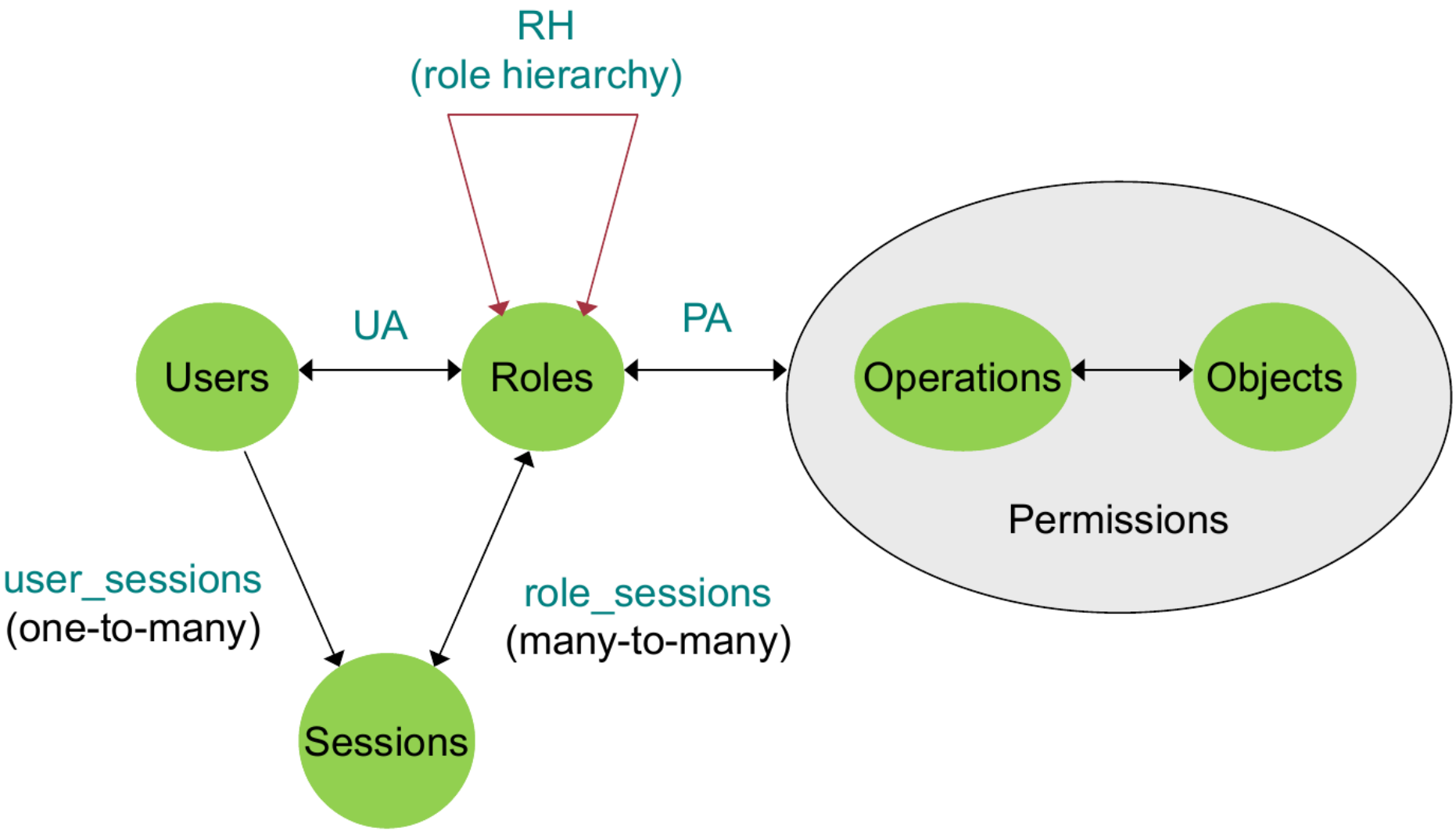
Role-Based Access Control (RBAC)

- Access control in organizations is based on *roles that individual users take on as part of the organization*
  - Access depends on function, not identity
  - Example:
    - Allison is bookkeeper for Math Dept
    - She has access to financial records
    - She leaves and Betty is hired as bookkeeper
    - The role of *bookkeeper* dictates access, not the identity of the individual
- A role is *a collection of permissions*

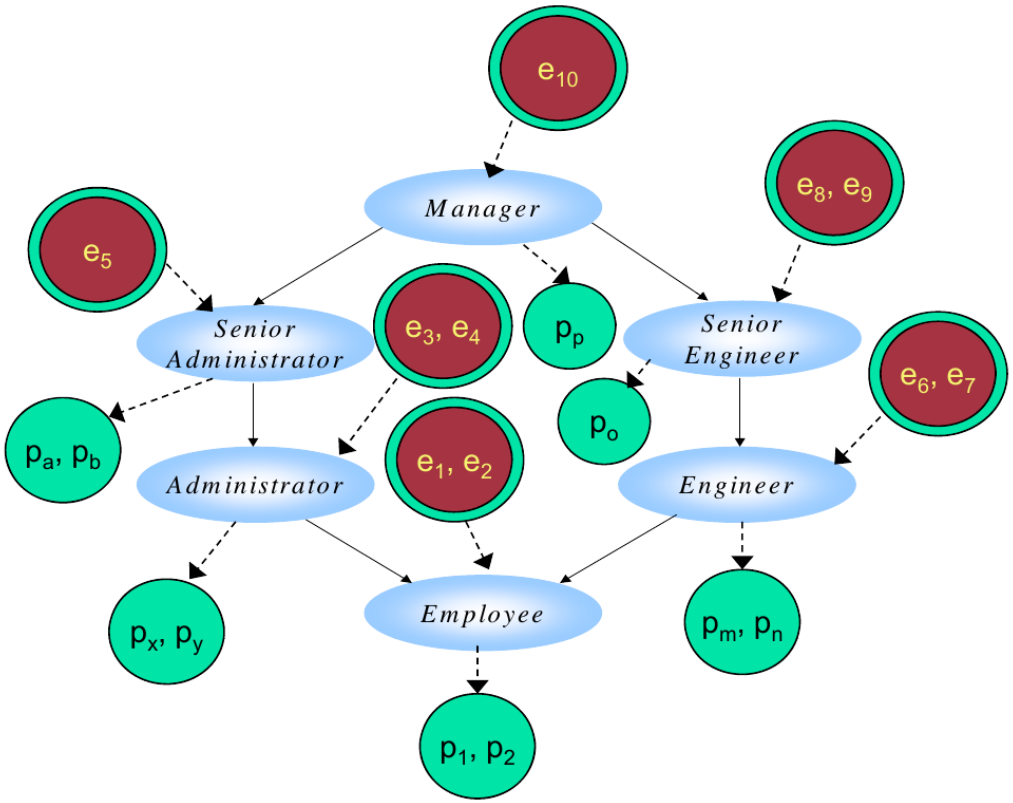
Core RBAC


Users
UA
Roles
PA
Operations
Objects
Permissions
user_sessions
(one-to-many)
role_sessions
(many-to-many)
Sessions

Hierarchical RBAC



Exercise 03c: Hierarchical RBAC



Calculate

- *authorized_users*(*Employee*)
- *authorized_users*(*Administrator*)
- *authorized_permissions*(*Administrator*)
- *authorized_permissions*(*SeniorEngineer*)

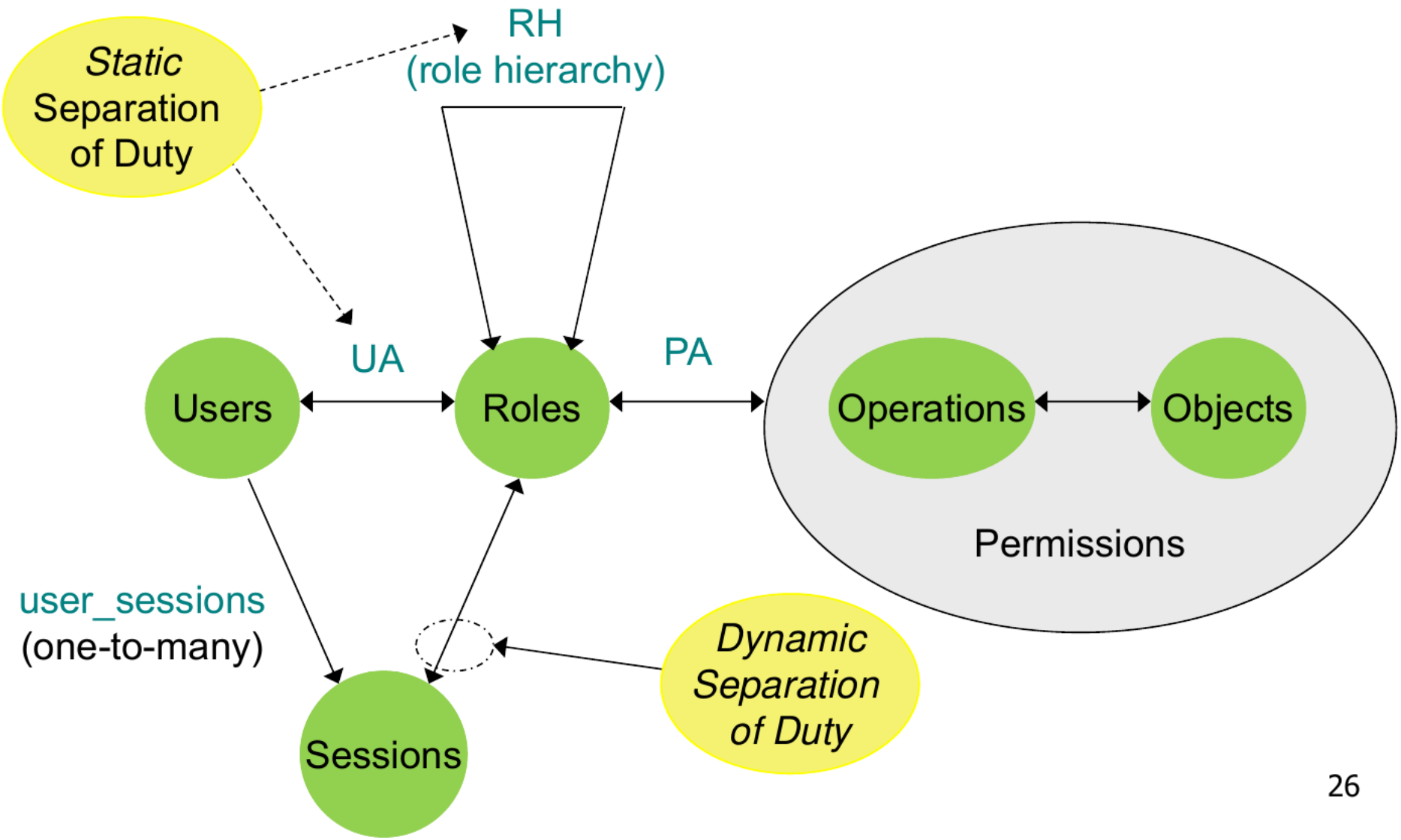
Questions:

1. *authorized_users*(*Administrator*) $\subseteq$ *authorized_users*(*Employee*)?
2. *authorized_permissions*(*Administrator*) $\cap$ *authorized_permissions*(*SeniorEngineer*) $= \emptyset$?

Separation of Duty

- Captures conflict of interest policies to restrict power of a single authority
  - Prevent Fraud
- Example
  - A single person should not be allowed to *approve a check* and *cash it*

Constrained RBAC: SSD RBAC & DSD RBAC



26

Attribute-Based Access Control (ABAC)

- Rule-based
  - A rule grants/denies access if a certain condition is true
- Express conditions on properties of subject, object, action, and environment

Relationship-Based Access Control (ReBAC)

- Emerged from Web 2.0 applications and online social networks
- More recent traction in industry for scalable authorization & zero-trust architecture
- Social relationships can be used as context to derive access control decision
  - e.g., your friends of friends
- But it can be used in other domains as well: healthcare, education, etc.
- Relationships can be between users and other entities